



TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH KHOA CÔNG NGHỆ THÔNG TIN

HỆ ĐIỀU HÀNH



ThS. Dương Minh Tuấn

0973887793

Email: dmtuan.vn

Quản lý hệ thống tập tin

- *Các yêu cầu lưu trữ thông tin*
- *Tập tin (File)*
- *Thư mục (directory / folder)*
- *Hệ thống quản lý tập tin*
- *Cấp phát không gian lưu trữ*
- *Quản lý không gian trống*

Các yêu cầu lưu trữ thông tin

■ Yêu cầu đối với phần cứng:

- ▶ **Persistence** (tính bền vững): thông tin lưu trữ trong thiết bị phải đảm bảo tính bền vững.
 - Đĩa từ tính: *băng từ, Floppy disk, Hard disk*
 - Đĩa quang: *CD, DVD...*
 - Chip bán dẫn: *NVRAM, NAND* (đĩa SSD).
- ▶ **Speed** (tốc độ): yêu cầu tốc độ truy xuất thông tin nhanh.
 - Tăng tốc độ xoay đĩa: 5.400 / 7.200 / 10.000... (rpm)
 - Tăng tốc truyền dẫn: SATA-1, SATA-2, SATA-3, SAS, SAS2, UBS3...
- ▶ **Size** (kích thước): thiết bị lưu trữ phải có kích thước lớn.
 - *Hard disk*: từ vài chục MB -> GB -> TB...
 - Đĩa quang: *CD* (640 MB) -> *DVD* (4.7GB)...

Các yêu cầu lưu trữ thông tin

■ Yêu cầu đối với Hệ điều hành:

- ▶ **Persistence** (tính bền vững): quản lý thông tin bền vững trong suốt vòng đời lưu trữ.
 - Tổ chức đơn vị lưu trữ theo chuỗi (*Cluster*)
 - Tổ chức lưu trữ theo hệ thống quản lý tập tin (*File System*)
- ▶ **Ease of use** (dễ sử dụng):
 - Tổ chức lưu trữ File theo thư mục (*Folder*).
 - Không phân biệt công nghệ phần cứng (ATA / SATA / SCSI / SAS...)
- ▶ **Sharing/Protection**: cho phép chia sẻ và bảo vệ.
 - Cho phép chia sẻ File cho hệ điều hành / máy tính khác truy cập.
 - Cho phép thiết lập các chế độ bảo vệ tập tin như: *Read only, Hidden...*

Tập tin (File)

■ Khái niệm tập tin (File):

- ▶ File là *chuỗi các bytes* thông tin được lưu trữ trên thiết bị đĩa.
- ▶ File có tính bền vững
- ▶ File được đặt tên (*file name*)
- ▶ File cho phép đọc (*read*), ghi (*write*) hoặc xóa (*delete*).

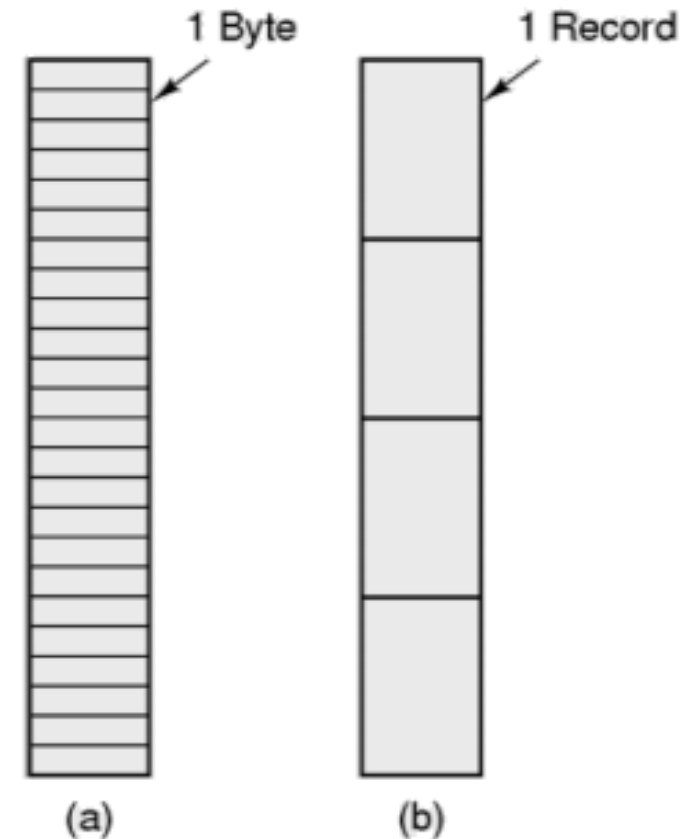
■ Phân loại tập tin:

- ▶ Phân loại theo nội dung thông tin lưu trữ:
 - File tài liệu, file thực thi, file hình, file âm thanh....
 - Thường dùng phần *ext* trong *tên file* để nhận dạng loại nội dung.
- ▶ Phân loại theo tính chất / thuộc tính:
 - File chỉ đọc, file ẩn, file cấm truy cập...
 - Trong UNIX: *regular file* (file thông thường), *directory* (file thư mục), *special device file* (file thiết bị).

Tập tin (File)

■ Cấu trúc tập tin:

- ▶ Tập tin không cấu trúc (tập tin thông thường):
 - Nội dung là chuỗi các bytes lưu trữ tuần tự.
- ▶ Tập tin có cấu trúc:
 - Nội dung là một dãy các mẫu tin (*record*)
 - Chiều dài mỗi *record* thường có kích thước cố định



Tập tin (File)

■ Phương pháp truy cập tập tin:

- ▶ Truy cập tuần tự (*sequential access*)
 - Truy xuất dữ liệu theo thứ tự, từng *byte* hoặc *record*.
- ▶ Truy cập bất kỳ (*random access*)
 - Truy xuất dữ liệu, byte hoặc record, tại vị trí bất kỳ
- ▶ Truy cập dùng khóa (*Key*)
 - Truy xuất một *record* dựa vào *key* của nó.



Tập tin (File)

■ Các tác vụ trên tập tin:

- ▶ *Create*: tác vụ **tạo** file mới
- ▶ *Write*: tác vụ **ghi** dữ liệu vào file tại vị trí con trỏ ghi
- ▶ *Read*: tác vụ **đọc** dữ liệu từ file tại vị trí con trỏ đọc
- ▶ *Delete*: **xóa** file
- ▶ *Open*: **mở** file trước khi sử dụng
- ▶ *Close*: **đóng** file sau khi sử dụng
- ▶ *Truncate*: giữ lại tất cả các thuộc tính của file, xóa hết nội dung.
- ▶ *Reposition*: Thiết lập con trỏ đọc/ghi đến vị trí do quá trình chỉ định

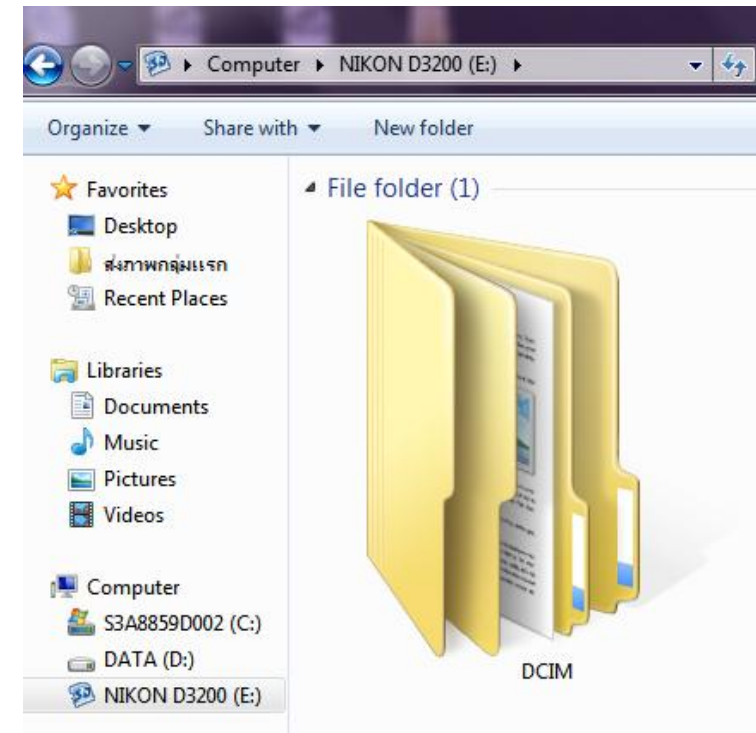
Thư mục (directory / folder)

■ Khái niệm thư mục:

- ▶ **Thư mục** (*directory, folder*) là một đối tượng tiện ích tạo ra bởi hệ điều hành dùng chứa files.
- ▶ Thư mục giúp người dùng dễ dàng tổ chức quản lý các file lưu trữ trong đĩa.
- ▶ Trong cấu trúc dạng **Tree**: Thư mục có thể chứa thư mục con.

■ Các tác vụ trên thư mục:

- ▶ Các tác vụ thông thường: *Create, Read, Delete, Open, Close*.
- ▶ *Link / Unlink*: thư mục liên kết.
 - Tạo *thư mục A* -> *thư mục B*.
 - => mở *thư mục A* cũng chính là mở *thư mục B*.



Thư mục (directory / folder)

■ Các loại cấu trúc thư mục:

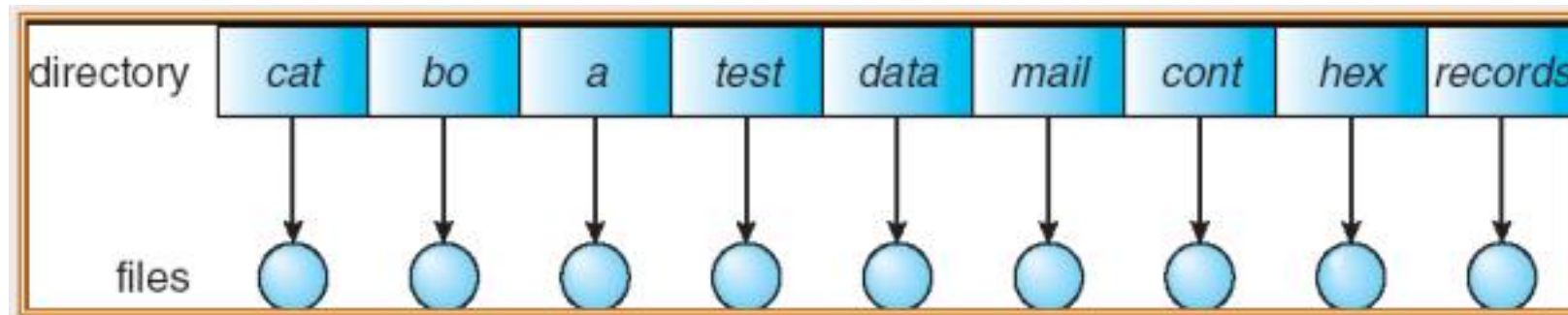
Có nhiều loại cấu trúc thư mục tùy theo Hệ điều hành

- ▶ ***Single-level directory*** (cấu trúc thư mục đơn cấp).
- ▶ ***Two-level directory*** (cấu trúc thư mục hai cấp).
- ▶ ***Tree-structured directory*** (cấu trúc thư mục hình cây).
- ▶ ***Acyclic-graph directory*** (cấu trúc thư mục đồ thị tuần hoàn).
- ▶ ***General graph directory*** (cấu trúc thư mục đồ thị tổng quát).

Thư mục (directory / folder)

■ Các loại cấu trúc thư mục:

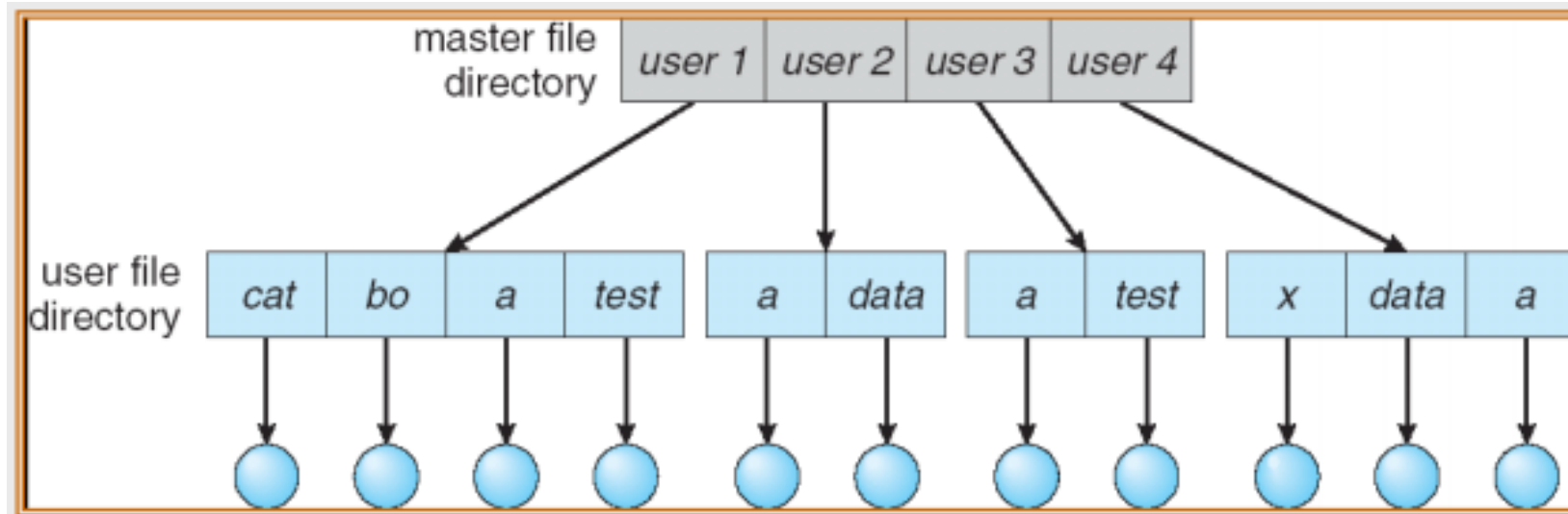
- ▶ **Single-level directory** (cấu trúc thư mục đơn cấp).
 - Chỉ có 1 cấp thư mục cho tất cả file lưu trữ trong hệ thống.
 - Hệ điều hành: CDC 6600, IBM MVS, PalmOS.



Thư mục (directory / folder)

■ Các loại cấu trúc thư mục:

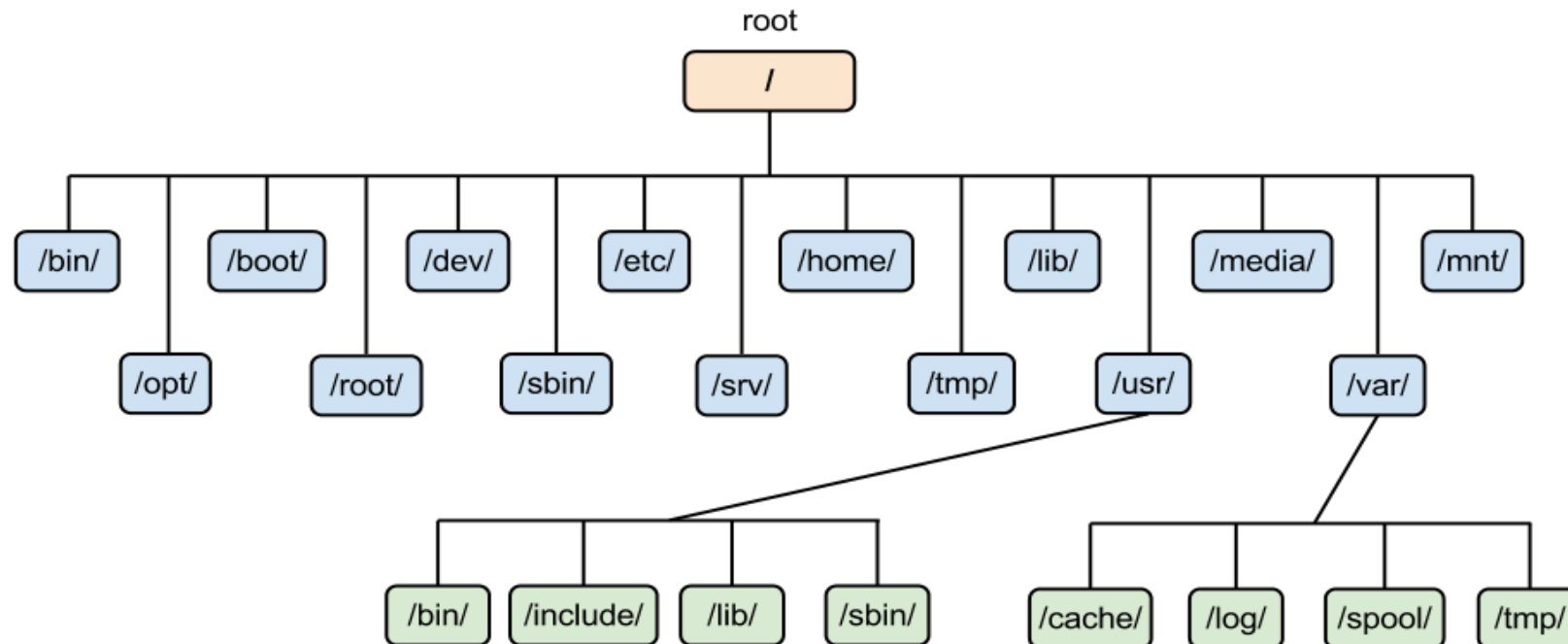
- ▶ **Two-level directory** (cấu trúc thư mục đa cấp).
 - Cải tiến từ *single-level directory*.
 - Phân chia theo người dùng.
 - Cho phép có file trùng tên



Thư mục (directory / folder)

■ Các loại cấu trúc thư mục:

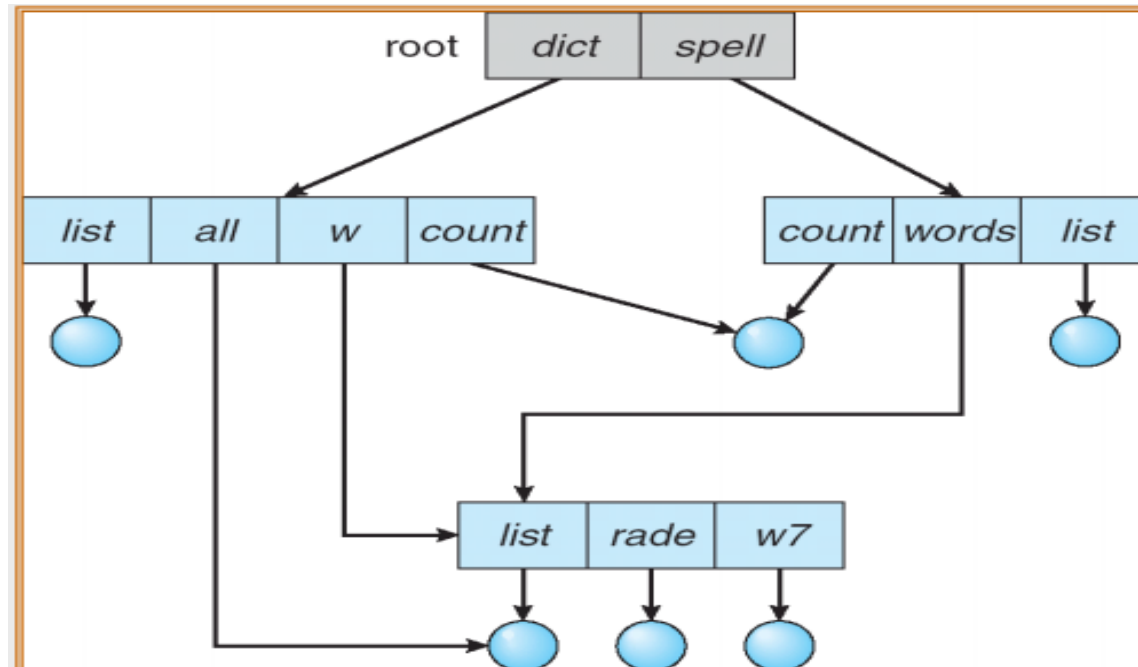
- ▶ **Tree-structured directory** (cấu trúc thư mục hình cây).
 - Dễ dàng phân nhóm tập tin.
 - Cho phép người dùng cùng 1 máy tính truy cập tập tin lẫn nhau.



Thư mục (directory / folder)

- **Các loại cấu trúc thư mục:**

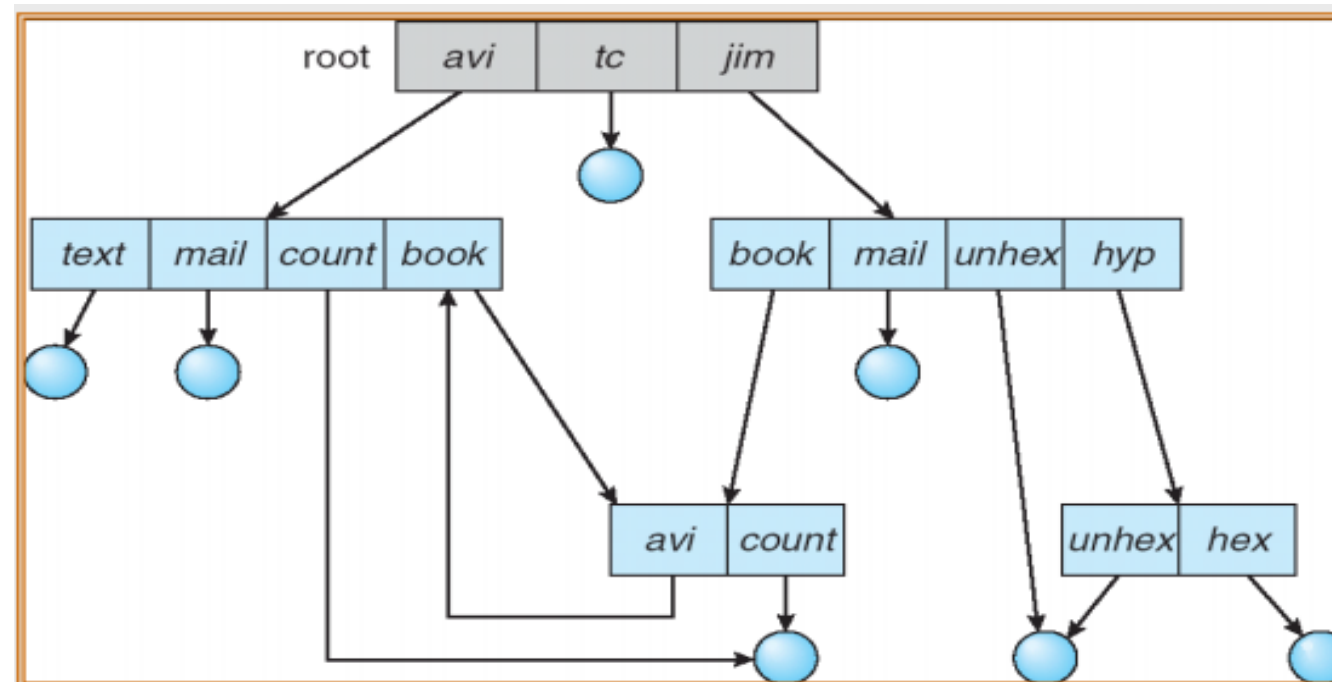
- ▶ ***Acyclic-graph directory*** (cấu trúc thư mục đồ thị tuần hoàn).
 - Cấu trúc tuần hoàn, không có chu trình.
 - Sử dụng các liên kết (*Symbolic Link*).
 - Cho phép 1 file (hay *thư mục con*) chia sẻ cho nhiều *thư mục cha*.



Thư mục (directory / folder)

■ Các loại cấu trúc thư mục:

- ▶ **General graph directory** (cấu trúc thư mục đồ thị tổng quát).
 - Cấu trúc đồ thị có thể chứa chu trình.
 - Có liên kết (*link*) tạo chu trình (min họa: *book* -> *avi* -> *book*)



Thư mục (directory / folder)

■ Đường dẫn (path)

- ▶ Khái niệm **Path**:
 - **Path**: là đường dẫn đến vị trí của *File* trong cấu trúc thư mục.
- ▶ Thư mục hiện hành (current directory, working directory)
- ▶ Đường dẫn tuyệt đối:
 - **Linux**: là đường dẫn được viết bắt đầu bởi *root directory* (thư mục **/**)
 - **Windows**: là đường dẫn bắt đầu từ tên *ổ đĩa* (ví dụ **C:**).
- ▶ Đường dẫn tương đối:
 - Là đường dẫn bắt đầu từ thư mục hiện hành.

Hệ thống quản lý tập tin

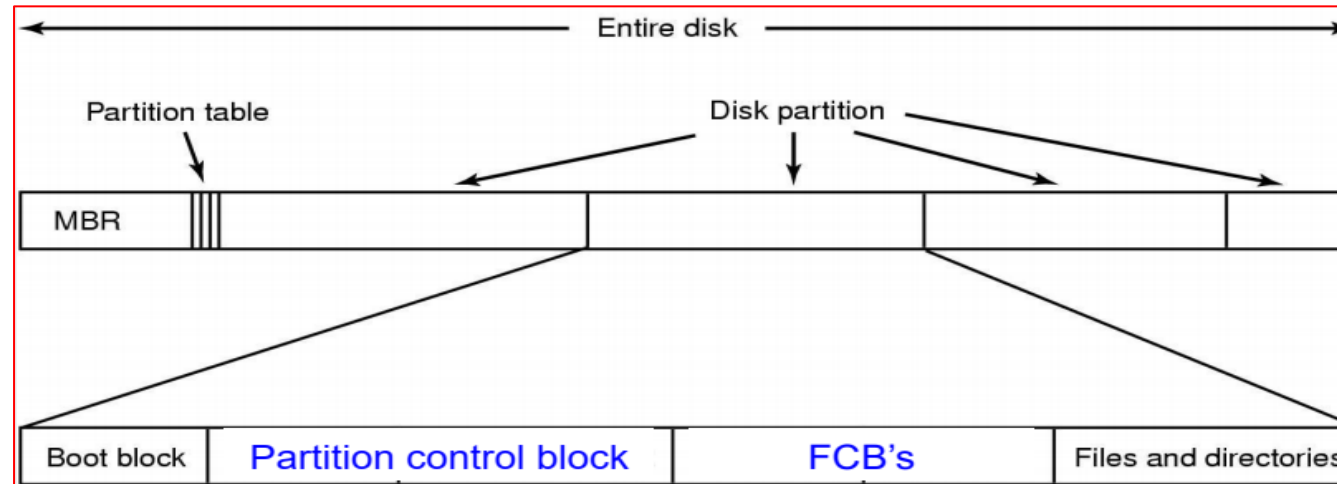
■ File System:

- ▶ **File system** là “hệ thống quản lý tập tin” của Hệ điều hành.
 - **Volume**: là một thực thể gồm 1 hay nhiều *partition* được quản lý bởi một *File System* của Hệ điều hành nào đó.
- ▶ Mỗi Hệ điều hành thường dùng *File System* riêng:
 - **MS-DOS**: dùng FAT (FAT-12, FAT-16, FAT-32)
 - **MS Windows**: dùng FAT và NTFS.
 - **Unix**: UFS, FHS
 - **Linux** (phát triển từ Unix): Ext2, Ext3, Ext4...
- ▶ Phương pháp quản lý:
 - Tổ chức không gian lưu trữ thành các **block** (hoặc **cluster**) bằng nhau, được đánh số địa chỉ.
 - Một bảng **File allocation table** (danh mục quản lý file) ghi nhận tên file, các vị trí các block lưu trữ, các thuộc tính...

Hệ thống quản lý tập tin

■ Layout tổ chức không gian lưu trữ trên đĩa:

- ▶ Chia không gian lưu trữ của *partition* thành các *block* bằng nhau
- ▶ **Partition Control Block (PCB):**
 - Chứa kích thước block, used blocks, free block...
 - Chứa địa chỉ con trỏ đến các blocks.
- ▶ **File control block (FCB):** lưu thông tin file, địa chỉ lưu trữ...
 - Hệ điều hành quản lý file thông qua **FCB**.



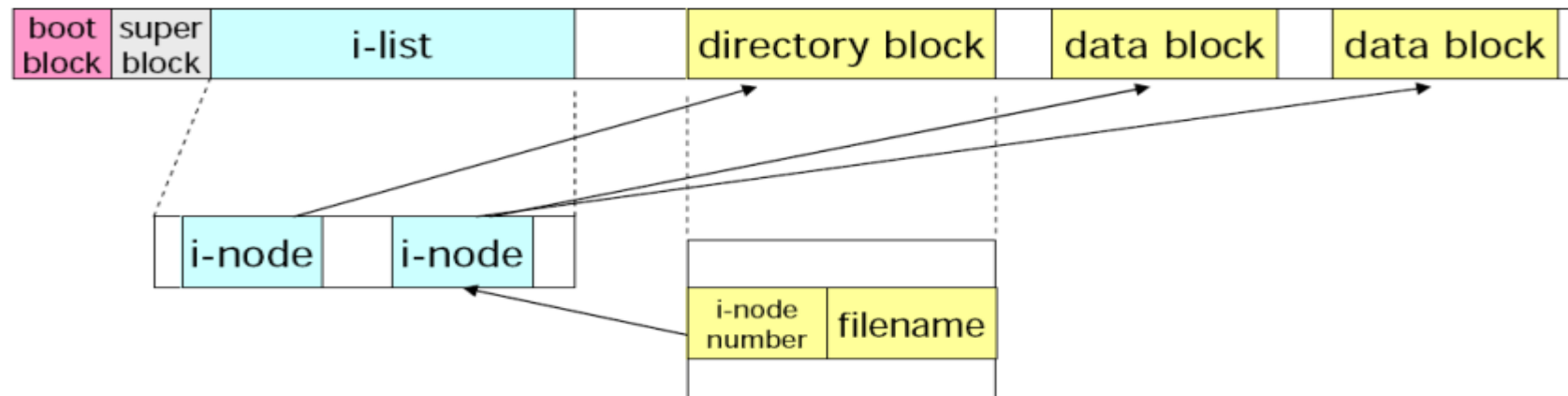
Hệ thống quản lý tập tin

Layout của các File System thông dụng

► Layout của *i-node File System* (**Unix**) và *Ext2 File System* (**Linux**) :

• PCB = **Super block**.

FCB = **i-node list**.



► Layout của *MS-DOS File System*

• PCB = **File Allocation Table (FAT)**

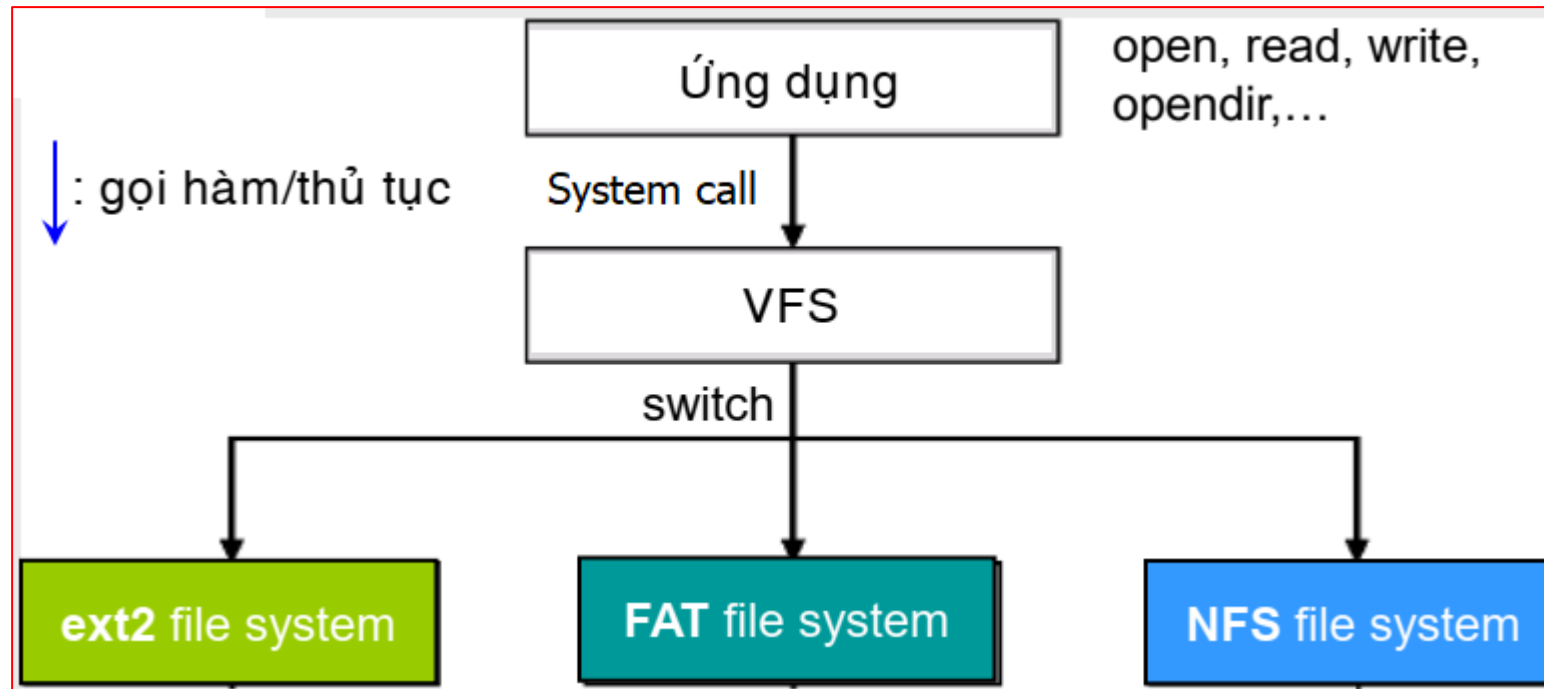
FCB = **Root directory**.



Hệ thống quản lý tập tin

■ Virtual File System (VFAT):

- ▶ **VFAT** là một thư viện dùng chung do Hệ điều hành tạo ra.
- ▶ **VFAT** hỗ trợ ứng dụng truy cập *Files* trên các ổ đĩa dùng những *File System* khác nhau.



Cấp phát không gian lưu trữ

■ Khái niệm và Mục tiêu:

▶ Khái niệm:

- Là giải pháp cấp phát không gian trên đĩa cho *1 File* hay *Fodder*.

▶ Mục tiêu:

- Sử dụng không gian lưu trữ hiệu quả nhất.
- Tốc độ truy xuất là nhanh nhất.

▶ Nguyên tắc:

- Gộp nhiều *sector* lại thành *block* (hay *Cluster*) để tăng hiệu suất.
- Cấp phát các *block* (hay *Cluster*) cho File / Folder.

▶ Phương pháp phổ biến:

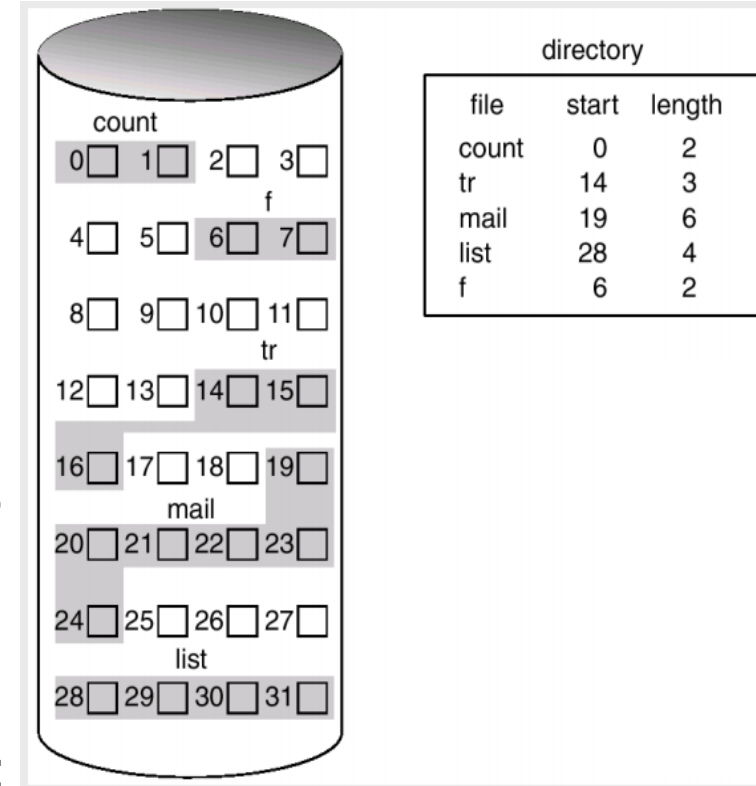
- Cấp phát liên tục (*continuos allocation*)
- Cấp phát theo danh sách liên kết (*linked list allocation*)
- Cấp phát dùng chỉ mục (*indexed allocation*)

Cấp phát không gian lưu trữ

■ Cấp phát liên tục (continuous allocation)

Cơ chế:

- Cơ chế này, các block tập tin được lưu trữ tại các block đĩa liên tục nhau. Tức là, nếu mỗi block đĩa là 1K thì một tập tin 50K sẽ được lưu trữ tại 50 block liên tiếp nhau trên đĩa.
- Cơ chế này đơn giản, dễ cài đặt và thời gian đọc tập tin giảm xuống đáng kể, vì hệ điều hành chỉ cần biết block đĩa đầu tiên chứa các block tập tin và tổng số block đĩa chứa tập tin là có thể tiến hành đọc nội dung của tập tin mà không cần dò tìm danh sách các block đĩa chứa nội dung của tập tin.
- Tuy nhiên, cơ chế này chỉ có thể được sử dụng với các tập tin có kích thước cố định, không thay đổi so với thời điểm tạo ra tập tin, hoặc với các tập tin mà hệ điều hành biết trước được kích thước tối đa của nó, trong trường hợp này hệ điều hành phải dự trữ block đĩa cho tập tin, điều này dễ dẫn đến tình trạng lãng phí trong việc sử dụng block đĩa.



Cấp phát không gian lưu trữ

■ Cấp phát liên tục (continuous allocation)

► Ưu điểm:

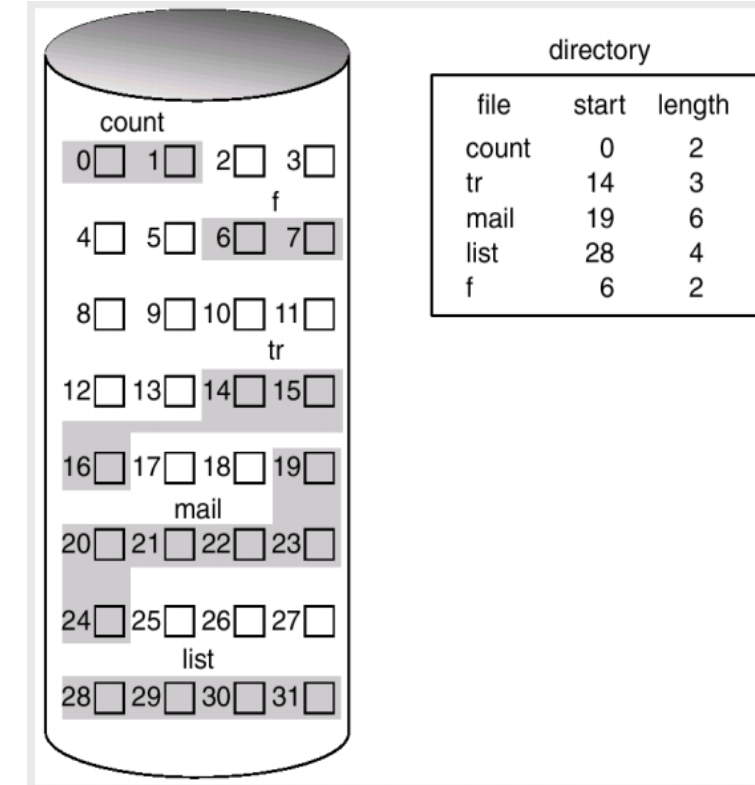
- Truy xuất nhanh, cài đặt đơn giản
- truy xuất ngẫu nhiên:
block start + block offset

► Nhược điểm:

- Phân mảnh ngoại.
- Vấn đề tạo file mới
- File tăng dung lượng

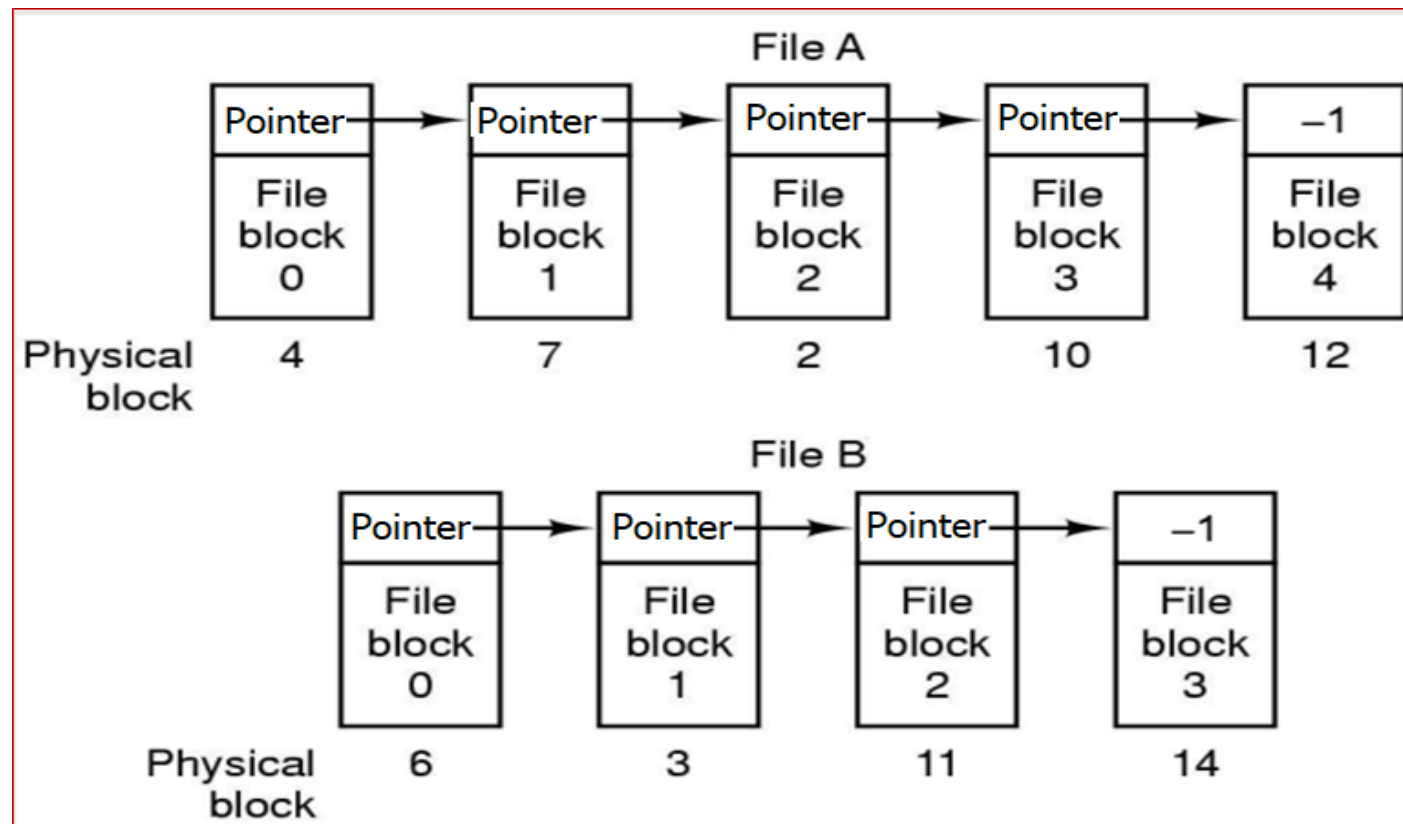
► Ứng dụng:

- Chuẩn **ISO-9660** dành cho CD-ROM.



Cấp phát không gian lưu trữ

- **Cấp phát theo danh sách liên kết (linked list allocation)**
 - ▶ *File* trong bảng thư mục chỉ cần trỏ đến *Start block* và *End block*.



Cấp phát không gian lưu trữ

■ Cấp phát theo danh sách liên kết (linked list allocation)

► Cơ chế:

- Cơ chế này sử dụng một danh sách liên kết để quản lý các block đĩa để chứa nội dung của một tập tin.
- Word đầu tiên của mỗi block đĩa được sử dụng như một con trỏ để trỏ đến block kế tiếp, trừ word của block cuối cùng được sử dụng để chứa tín hiệu báo kết thúc danh sách của một tập tin, phần còn lại của block đĩa dùng để chứa nội dung của tập tin.
- Trong trường hợp này kích thước của block đĩa phải lớn hơn kích thước của block tập tin 1 word.

Cấp phát không gian lưu trữ

■ Cấp phát theo danh sách liên kết (linked list allocation)

► Ưu điểm

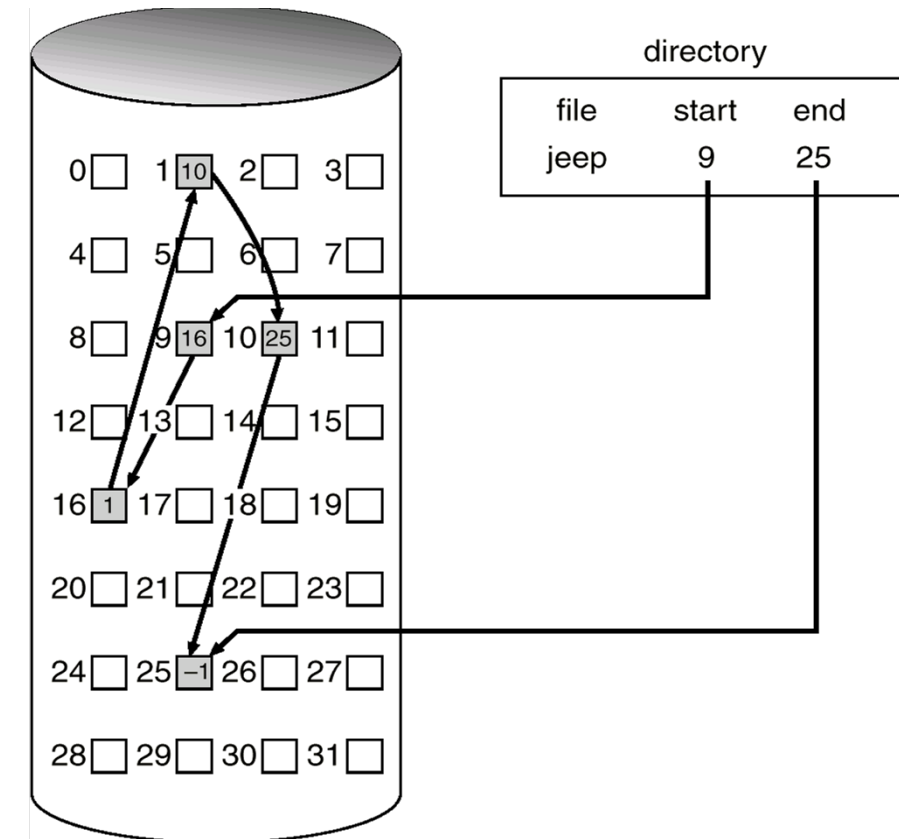
- Dễ dàng thêm block cho file khi cần
- Quản lý không gian trống bằng danh sách liên kết
- Không có phân mảnh ngoại và khai thác tối đa không gian đĩa

► Nhược điểm

- Tốc độ truy cập file chậm, chỉ truy xuất hiệu quả đối với *sequential-access file* (tập tin truy cập tuần tự).
- Tốn không gian lưu trữ *pointers*
- Độ tin cậy: *pointer* trong block có thể bị hỏng.

► Ứng dụng:

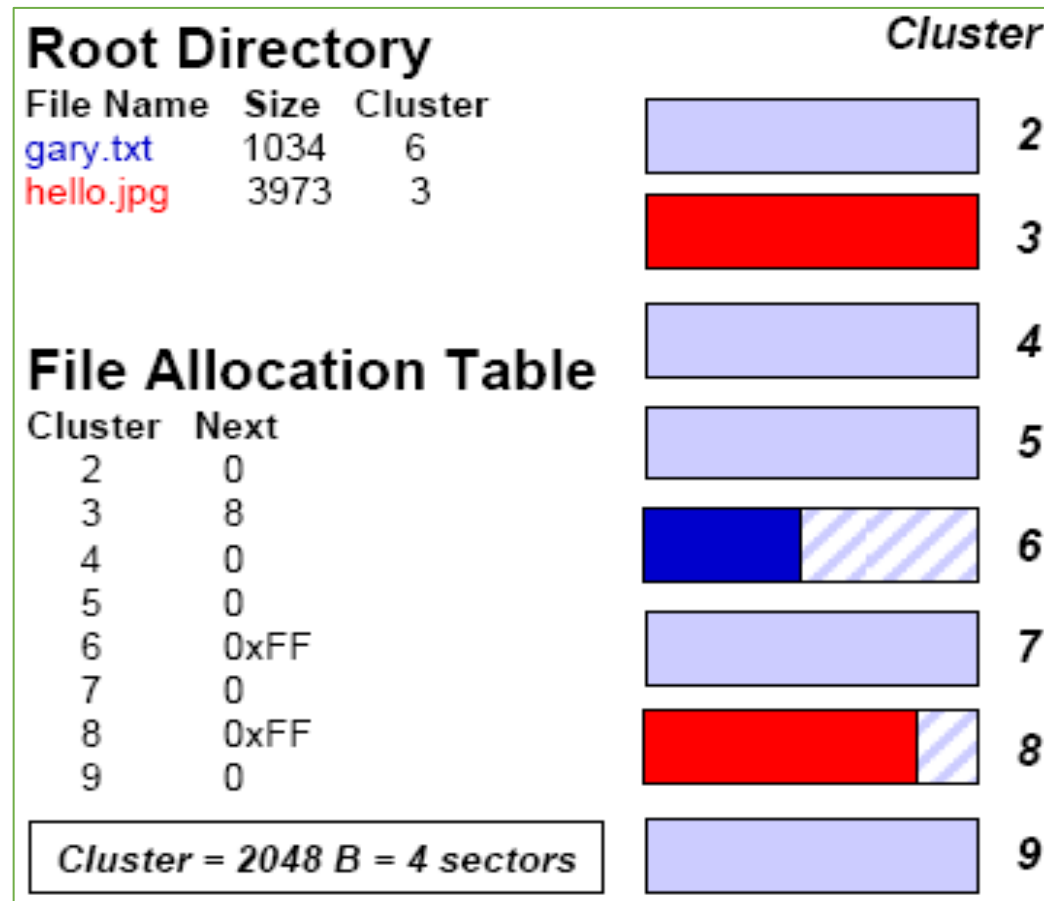
- *FAT File system* của MS-DOS



Hệ thống quản lý tập tin

■ Cấp phát theo danh sách liên kết - *FAT File System*:

- ▶ Được *Microsoft* phát triển và sử dụng cho *MS-DOS*, *Windows 95*...



Cấp phát không gian lưu trữ

■ Cấp phát dùng chỉ mục (indexed allocation)

► Phương pháp:

- *Block* đầu tiên chứa các chỉ mục (*index*) nơi chứa các *block* khác chứa nội dung file.
- *Root directory* trỏ đến *indexed block*.

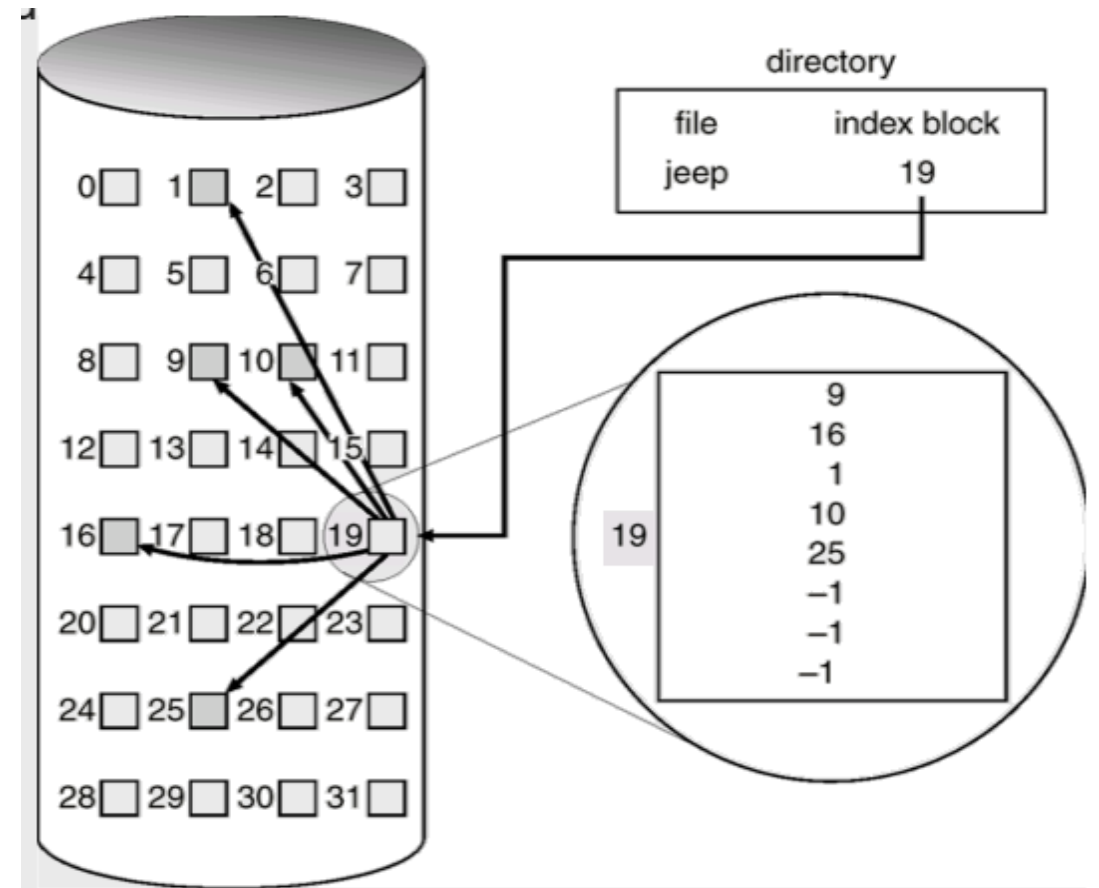
► Ưu điểm

- Phù hợp với kiểu *Random* và *Sequential access*
- Không có phân mảnh ngoại

► Nhược điểm

- Tốn không gian cho bảng *index block*.

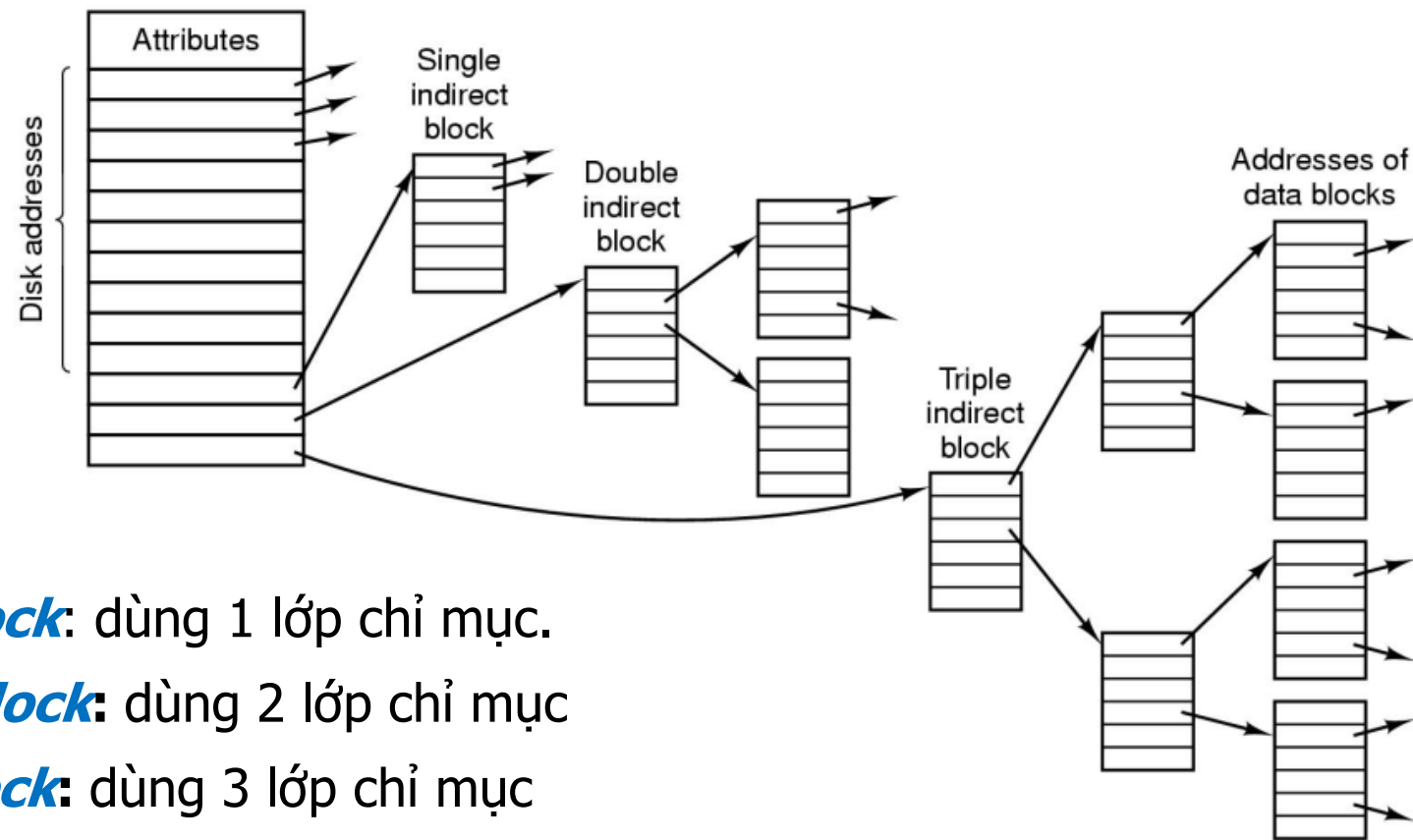
► Ứng dụng: *i-node FS*.



Cấp phát không gian lưu trữ

■ Cấp phát dùng chỉ mục: *i-node* và *Ext2 file system*

- ▶ *i-node file system* dùng nhiều cấu trúc chỉ mục khác nhau tương ứng theo kích thước file từ nhỏ đến lớn



- ▶ *Single indirect block*: dùng 1 lớp chỉ mục.
- ▶ *Double-indirect block*: dùng 2 lớp chỉ mục
- ▶ *Triple-indirect block*: dùng 3 lớp chỉ mục

Cấp phát không gian lưu trữ

■ Cấp phát dùng chỉ mục: *i-node* và *Ext2 file system*

- ▶ Mô tả *bảng thư mục* (*Directory i-node*):
 - *File name*: tên tập tin.
 - *i-node #*: vị trí của *block* lưu *bảng chỉ mục* (*indexed block*).
- ▶ Mô tả *bảng chỉ mục* (*i-node*)
 - Mỗi *File* được cấp 1 *bảng chỉ mục* (*i-node file*) riêng.
 - Các trường thuộc tính file: *permissions*, *size*, *modified time*...
 - *Direct block* (**0** và **1**): trỏ vị trí 2 *block* nội dung đầu tiên.
 - *Indirect-block #*, *double-indirect #*, *triple-indirect #*: trỏ vị trí lưu trữ các bảng chỉ mục mở rộng tương ứng.
 - *Direct block #*: trỏ vị trí (số hiệu khối) của các block nội dung.

Quản lý không gian trống

- **Các phương pháp:**
 - ▶ *Bit vector (bit map)*
 - ▶ *Linked list*
 - ▶ *Grouping*
 - ▶ *Counting*

Quản lý không gian trống

■ Phương pháp Bit vector (bit map)

- ▶ Nguyên tắc: dùng dãy bít để xác định block là *free* hay *used*.

$$\text{bit}[i] = \begin{cases} 0 \Rightarrow \text{block } i \text{ còn trống} \\ 1 \Rightarrow \text{block } i \text{ đã được cấp} \end{cases}$$

- ▶ **Ví dụ:** *bit vector* **00111100**...

- block 0, 1, 6, 7: free - block 2, 3, 4, 5: used.

- ▶ **Ưu điểm:**

- Đơn giản và hiệu quả khi cần tìm khối trống đầu tiên hoặc chuỗi khối trống liên tục
- Thao tác trên bit

- ▶ **Khuyết:**

- Cần không gian lưu trữ. Ví dụ

Quản lý không gian trống

■ Phương pháp Linked list

▶ Phương pháp

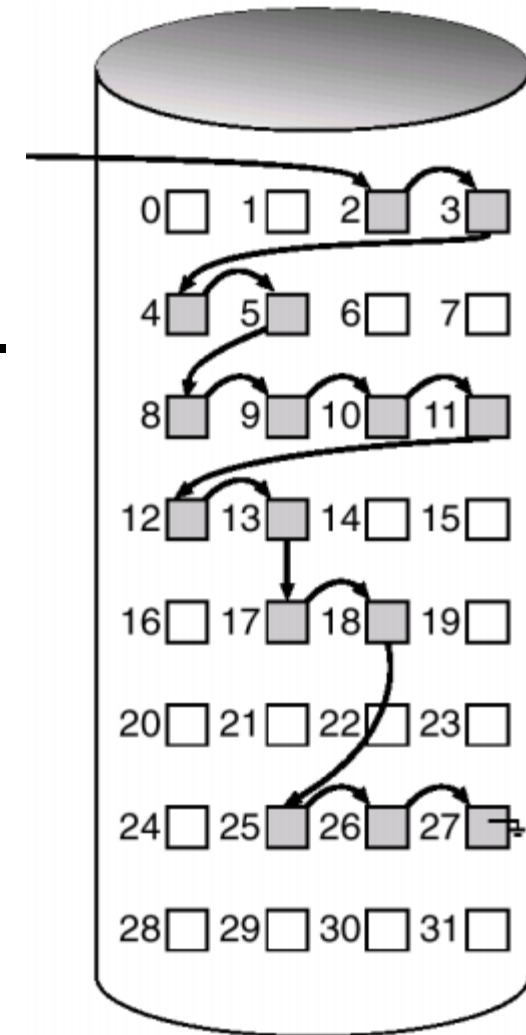
- Liên kết các khối trống với nhau
- Chỉ cần giữ con trỏ đến khối nhớ trống đầu tiên.

▶ Ưu:

- Ít lãng phí không gian đĩa

▶ Nhược:

- Không hiệu quả.
- Trong trường hợp xấu nhất phải duyệt lại toàn bộ đĩa để tìm không gian trống liên tục



Quản lý không gian trống

■ Phương pháp Grouping và Counting:

▶ Phương pháp *Grouping*

- Địa chỉ của n khối trống được lưu trong khối trống đầu tiên.
- Khối nhớ thứ n chứa địa chỉ của n khối nhớ trống kế tiếp.

▶ Phương pháp *Counting*

- Tổ chức bảng chỉ mục
- Mỗi entry: địa chỉ của khối trống đầu tiên trong nhóm khối trống liên tục và một số đếm số lượng khối trống.
- Có thể cấp phát hoặc thu hồi đồng thời nhiều khối nhớ liên tục

